

Aim: Naive Bayes' Classifier

Implement the Naive Bayes' algorithm for classification. Train a Naive Bayes' model using a given dataset and calculate class Probabilities. Evaluate the accuracy of the model on test data and analyze the results.

```
# Importing the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Importing the dataset
dataset = pd.read_csv('iris.csv')
X = dataset.iloc[:, [0, 3]].values
y = dataset.iloc[:, -1].values

# Splitting the dataset into the Training set and Test set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size =
0.20, random_state = 0)

# Feature Scaling
#Since our dataset containing character variables
#we have to encode it using LabelEncoder
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# Training the Naive Bayes model on the Training set
from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB()
classifier.fit(X_train, y_train)

# Predicting the Test set results
y_pred = classifier.predict(X_test)
print("Predicted Test Results : ",y_pred)
print("~"*20)

# Making the Confusion Matrix
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
ac = accuracy_score(y_test,y_pred)
print("Model Accuracy : ",ac*100,"%")
print("~"*20)
cm = confusion_matrix(y_test, y_pred)
print("Model Confusion Matrix : ")
print(cm)
```

```
Predicted Test Results : ['Iris-virginica' 'Iris-versicolor' 'Iris-setosa' 'Iris-virginica'
'Iris-setosa' 'Iris-virginica' 'Iris-setosa' 'Iris-versicolor'
'Iris-versicolor' 'Iris-versicolor' 'Iris-virginica' 'Iris-versicolor'
'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor' 'Iris-setosa'
'Iris-versicolor' 'Iris-versicolor' 'Iris-setosa' 'Iris-setosa'
'Iris-virginica' 'Iris-versicolor' 'Iris-setosa' 'Iris-setosa'
'Iris-virginica' 'Iris-setosa' 'Iris-setosa' 'Iris-versicolor'
'Iris-versicolor' 'Iris-setosa']
~~~~~
Model Accuracy : 100.0 %
~~~~~
Model Confusion Matrix :
[[11  0  0]
 [ 0 13  0]
 [ 0  0  6]]
```

#The problem statement is to classify patients as diabetic or non-diabetic. Database:

[Pima Indians Diabetes Database](#)

The datasets had several different medical predictor features and a target that is 'Outcome'. Predictor variables include the number of pregnancies the patient has had, their BMI, insulin level, age, and so on.

STEPS-

- Initially, all the necessary libraries are imported like numpy, pandas, train-test_split, GaussianNB, metrics.
- Since it is a data file with no header, we will supply the column names that have been obtained from the above URL.
- Created a python list of column names called "names".
- Initialized predictor variables and the target that is X and Y respectively.
- Transformed the data using StandardScaler.
- Split the data into training and test sets.
- Created an object for GaussianNB.
- Fitted the data in the model to train it.
- Made predictions on the test set and stored it in a 'predictor' variable.
- Result Diagnosis

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn import metrics

print("Libraries imported")
```

```
#Since it is a data file with no header,
#we will supply the column names that have been obtained from the above
URL
colnames = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age',
'class']
pima_df = pd.read_csv("pima-indians-diabetes.data.csv", names=colnames)
print("Dataset Loaded")
```

```
colnames = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age',
'class']

pima_df = pd.read_csv("pima-indians-diabetes.data.csv", names=colnames)

print(pima_df.head())
```

The screenshot shows a Jupyter Notebook with the following code and output:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn import metrics

print("Libraries imported")
```

Output: Libraries imported

```
#Since it is a data file with no header,
#we will supply the column names that have been obtained from the above URL
colnames = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
pima_df = pd.read_csv("pima-indians-diabetes.data.csv", names=colnames)
print("Dataset Loaded")
```

Output: Dataset Loaded

```
colnames = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']

pima_df = pd.read_csv("pima-indians-diabetes.data.csv", names=colnames)

print(pima_df.head())
```

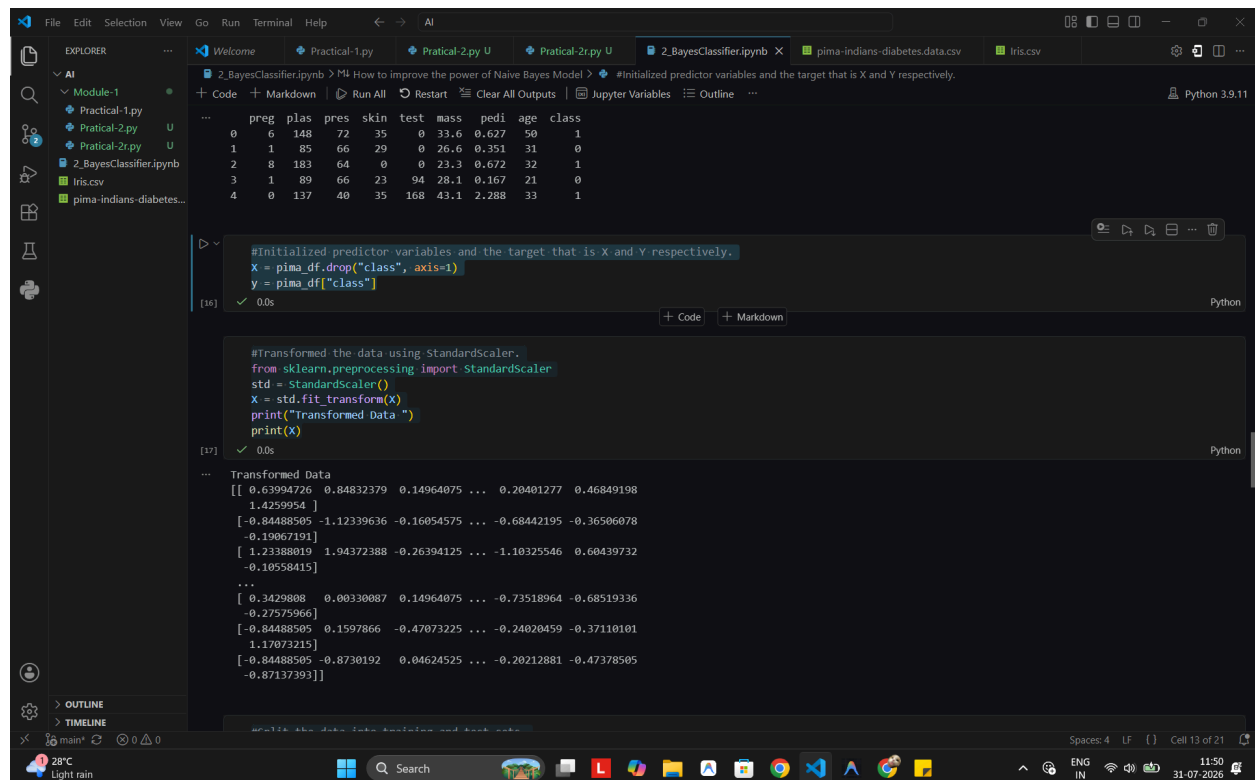
Output:

	preg	plas	pres	skin	test	mass	pedi	age	class
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Sheth L.U.J & Sir M.V College
Subject:AI

```
#Initialized predictor variables and the target that is X and Y  
respectively.  
X = pima_df.drop("class", axis=1)  
y = pima_df["class"]
```

```
#Transformed the data using StandardScaler.  
from sklearn.preprocessing import StandardScaler  
std = StandardScaler()  
X = std.fit_transform(X)  
print("Transformed Data ")  
print(X)
```



The screenshot shows a Jupyter Notebook with two code cells. The first cell initializes predictor variables (X) and the target variable (y) from the Pima Indians Diabetes dataset. The second cell uses StandardScaler to transform the data. The output of the second cell shows the transformed data as a NumPy array.

```
#Initialized predictor variables and the target that is X and Y respectively.  
X = pima_df.drop("class", axis=1)  
y = pima_df["class"]
```

```
#Transformed the data using StandardScaler.  
from sklearn.preprocessing import StandardScaler  
std = StandardScaler()  
X = std.fit_transform(X)  
print("Transformed Data ")  
print(X)
```

Transformed Data

```
[[ 0.63994726  0.84832379  0.14964075 ...  0.20481277  0.46849198  
  1.4259954 ]  
 [-0.84488505 -1.12339636 -0.16054575 ... -0.68442195 -0.36506078  
 -0.19667191]  
 [ 1.23388019  1.94372388 -0.26394125 ... -1.10325546  0.60439732  
 -0.10558415]  
 ...  
 [ 0.3429808  0.00330087  0.14964075 ... -0.73518964 -0.68519336  
 -0.27575966]  
 [-0.84488505  0.1597866  -0.47073225 ... -0.24020459 -0.37110101  
  1.17073215]  
 [-0.84488505 -0.8730192  0.04624525 ... -0.20212881 -0.47378505  
 -0.87137393]]
```

```
#Split the data into training and test sets.  
test_size = 0.30 # taking 70:30 training and test set  
seed = 7 # Random numbmer seeding for reapeatability of the code  
X_train, X_test, y_train, y_test = train_test_split(X, y,  
test_size=test_size, random_state=seed)
```

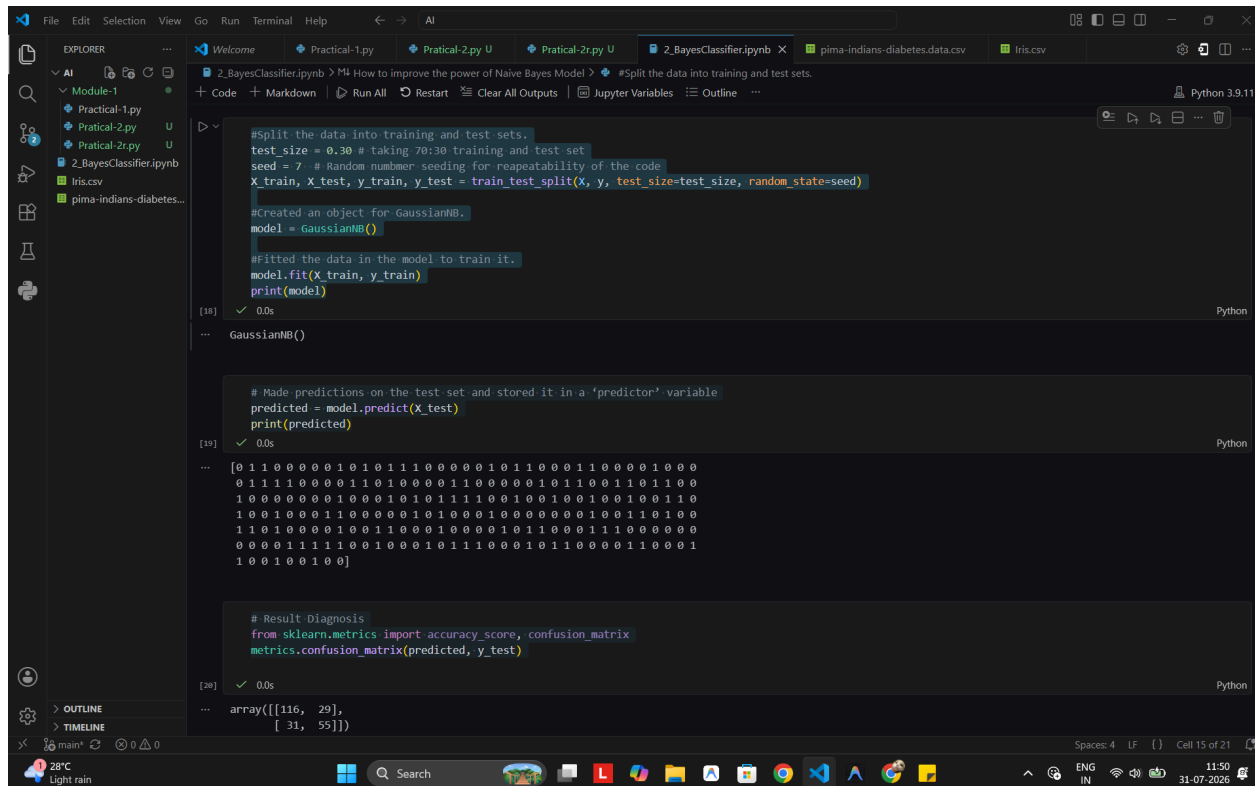
```
#Created an object for GaussianNB.  
model = GaussianNB()
```

```
#Fitted the data in the model to train it.  
model.fit(X_train, y_train)  
print(model)
```

```
# Made predictions on the test set and stored it in a 'predictor' variable  
predicted = model.predict(X_test)  
print(predicted)
```

```
# Result Diagnosis  
from sklearn.metrics import accuracy_score, confusion_matrix  
metrics.confusion_matrix(predicted, y_test)
```

Sheth L.U.J & Sir M.V College
Subject: AI



The screenshot displays a Jupyter Notebook titled '2_BayesClassifier.ipynb' within the Visual Studio Code editor. The notebook is open to a cell containing Python code for a Gaussian Naive Bayes classifier. The code includes data splitting, model training, prediction, and a result diagnosis using sklearn metrics. The output shows a list of predicted values and a confusion matrix.

```
#Split the data into training and test sets.
test_size = 0.30 # taking 70:30 training and test set
seed = 7 # Random number seeding for repeatability of the code
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size, random_state=seed)

#created an object for GaussianNB.
model = GaussianNB()

#Fitted the data in the model to train it.
model.fit(X_train, y_train)
print(model)

[18] ✓ 0.0s Python
GaussianNB()

# Made predictions on the test set and stored it in a 'predictor' variable
predicted = model.predict(X_test)
print(predicted)

[19] ✓ 0.0s Python
[0 1 1 0 0 0 0 0 1 0 1 0 1 1 1 0 0 0 0 0 1 0 1 1 0 0 0 1 1 0 0 0 0 1 0 0 0
0 1 1 1 1 0 0 0 0 1 1 0 1 0 0 0 0 1 1 0 0 0 0 0 1 0 1 1 0 0 1 1 0 1 1 0 0
1 0 0 0 0 0 0 0 1 0 0 0 1 0 1 0 1 1 1 1 0 0 1 0 0 1 0 0 1 0 0 1 1 0
1 0 0 1 0 0 0 1 1 0 0 0 0 0 1 0 1 0 0 0 1 0 0 0 0 0 0 0 1 0 0 1 1 0 1 0 0
1 1 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 1 0 1 1 0 0 0 1 1 1 0 0 0 0 0
0 0 0 0 1 1 1 1 0 0 1 0 0 0 1 0 1 1 1 0 0 0 1 0 1 1 0 0 0 0 1 1 0 0 0 1
1 0 0 1 0 0 1 0 0]

# Result Diagnosis
from sklearn.metrics import accuracy_score, confusion_matrix
metrics.confusion_matrix(predicted, y_test)

[20] ✓ 0.0s Python
array([[116, 29],
       [ 31, 55]])
```

```
#https://scikit-learn.org/stable/modules/model_evaluation.html
model_score = model.score(X_test, y_test)
model_score
```

```
#https://towardsdatascience.com/predict-vs-predict-proba-scikit-learn-bdc45daa5972
#https://scikit-learn.org/stable/search.html?q=predict_proba
```

```
y_predictProb = model.predict_proba(X_test)
print(y_predictProb)
```

The screenshot shows a Jupyter Notebook environment. The top bar indicates the file is '2_BayesClassifier.ipynb' and the kernel is 'Python 3.9.11'. The left sidebar shows the file explorer with 'Module-1' expanded, containing 'Practical-1.py', 'Practical-2.py', and '2_BayesClassifier.ipynb'. The main area displays a code cell with the following content:

```
#https://scikit-learn.org/stable/search.html?q=predict_proba
y_predictProb = model.predict_proba(X_test)
print(y_predictProb)
```

The output of the cell is a list of 2D probability arrays for each sample in the test set. The output is truncated, showing the first 20 and last 4 samples. The first sample's output is:

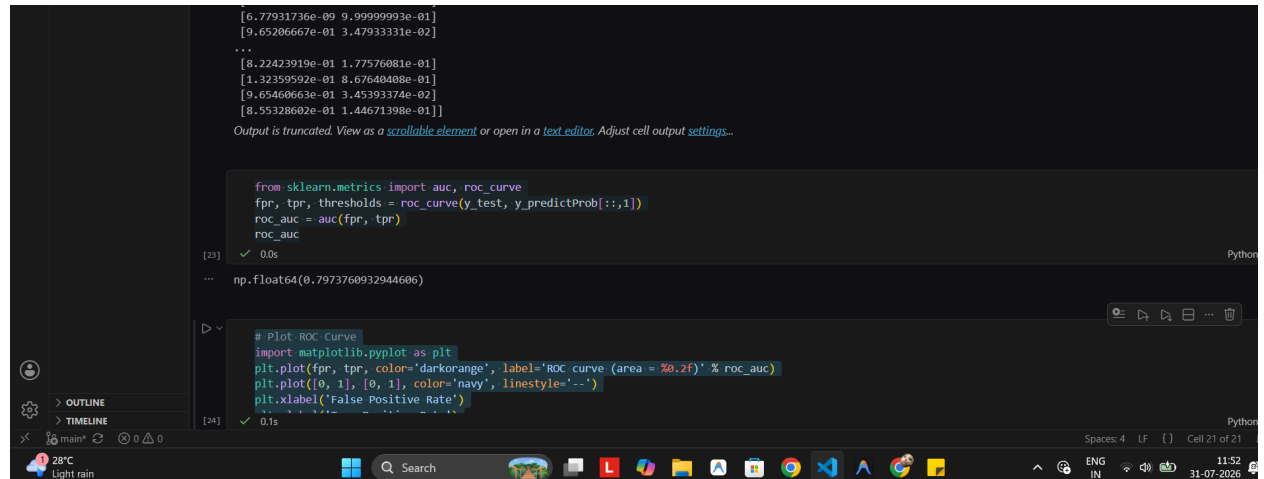
```
[[9.86158573e-01 1.38414272e-02]
 [3.12380377e-02 9.68761962e-01]
 [5.40635227e-02 9.45936477e-01]
 [9.65065125e-01 3.49348746e-02]
 [7.50736253e-01 2.49263747e-01]
 [6.17355267e-01 3.82644733e-01]
 [9.73541945e-01 2.64580550e-02]
 [9.49535030e-01 5.04649697e-02]
 [1.97506776e-03 9.98024932e-01]
 [9.33568191e-01 6.64318089e-02]
 [3.96675687e-02 9.60332431e-01]
 [9.25723305e-01 7.42766952e-02]
 [4.43087076e-02 9.55691292e-01]
 [3.24714802e-01 6.75285190e-01]
 [4.10899828e-01 5.80100172e-01]
 [7.58424433e-01 2.41575567e-01]
 [8.25176629e-01 1.74823371e-01]
 [9.43192727e-01 5.68072733e-02]
 [8.52330066e-01 1.47669934e-01]
 [9.41638552e-01 5.83614478e-02]
 [2.14629528e-03 9.92350848e-01]
 [8.07450257e-01 1.92549743e-01]
 [7.64915233e-03 9.92350848e-01]
 [6.77931736e-09 9.99999993e-01]
 [9.65206667e-01 3.47933331e-02]
 ...
 [8.22423919e-01 1.77576081e-01]
 [1.32359592e-01 8.67640408e-01]
 [9.65460663e-01 3.45393374e-02]
 [8.55328602e-01 1.44671398e-01]]
```

Below the output, a message states: "Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output settings."

The bottom status bar shows the file is 'main', the kernel is 'Python 3.9.11', and the cell is 'Cell 19 of 21'. The system tray at the bottom indicates a temperature of 28°C, light rain, and the date 31-07-2026.

Sheth L.U.J & Sir M.V College
Subject:AI

```
from sklearn.metrics import auc, roc_curve
fpr, tpr, thresholds = roc_curve(y_test, y_predictProb[:,1])
roc_auc = auc(fpr, tpr)
roc_auc
```



The screenshot shows a Jupyter Notebook interface with two code cells. The first cell calculates the ROC AUC value, and the second cell plots the ROC curve. The output of the first cell is a truncated list of thresholds and a single AUC value. The second cell's output is a plot of the ROC curve.

```
[6.77931736e-09 9.99999993e-01]
[9.65206667e-01 3.47933331e-02]
...
[8.22423919e-01 1.77576081e-01]
[1.32259592e-01 8.67640408e-01]
[9.65460663e-01 3.45393374e-02]
[8.55328602e-01 1.44671398e-01]]
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

```
from sklearn.metrics import auc, roc_curve
fpr, tpr, thresholds = roc_curve(y_test, y_predictProb[:,1])
roc_auc = auc(fpr, tpr)
roc_auc
```

[23] ✓ 0.0s Python

```
np.float64(0.7973760932944606)
```

```
# Plot ROC Curve
import matplotlib.pyplot as plt
plt.plot(fpr, tpr, color='darkorange', label='ROC curve (area = %0.2f)' % roc_auc)
plt.plot([0, 1], [0, 1], color='navy', linestyle='--')
plt.xlabel('False Positive Rate')
```

[24] ✓ 0.1s Python

The bottom of the image shows a Windows taskbar with the date 31-07-2026 and time 11:52.

Sheth L.U.J & Sir M.V College
Subject:AI

```
# Plot ROC Curve
import matplotlib.pyplot as plt
plt.plot(fpr, tpr, color='darkorange', label='ROC curve (area = %0.2f)' %
roc_auc)
plt.plot([0, 1], [0, 1], color='navy', linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic')
plt.legend(loc="lower right")
```

